

Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Name: Giorgos Aridas Id: 1115202200011

Contents

1 Data Preprocessing and Initial Event Replayer Implementation **2**
1.1 Data Source and Preprocessing Pipeline 2
1.2 The Replayer Thread and Shared Memory 3
1.3 Due Time Calculation and Time Synchronization 4
1.3.1 Mathematical Formulation 4
1.3.2 Clock Selection: Monotonic vs. Real-Time 5
1.3.3 Latency Evaluation Metric 5

1 Data Preprocessing and Initial Event Replayer Implementation

1.1 Data Source and Preprocessing Pipeline

The main data source utilized for this project is StatsBomb. StatsBomb is an open online database that provides detailed records of football matches in `.json` format. Our initial task was to filter the necessary information required for the implementation of the final system. The extracted fields are:

- milliseconds (from the start of the game)
- clock time (view of the clock)
- x coordinate of the ball
- y coordinate of the ball
- description of the event

```
{
  "id": "ba455dc1-b4f0-4c31-bde7-7c338f8a90e1",
  "index": 786,
  "period": 1,
  "timestamp": "00:19:21.427",
  "minute": 19,
  "second": 21,
  "type": {
    "id": 30,
    "name": "Pass"
  },
  "player" : {
    "id" : 6826,
    "name" : "Cl ment Lenglet"
  },
  "location": [61.0, 40.1]
}
```

Listing 1: Sample raw StatsBomb event JSON

Listing 1 illustrates a representative fragment of a raw event encoded in JSON format, as provided by the StatsBomb dataset. JSON (JavaScript Object Notation) is a lightweight, structured data-interchange format. As shown in the snippet, the JSON structure embeds key metadata regarding each action; for instance, it specifies an event type of 'Pass' performed by a player at the precise field coordinates denoted by the x and y values. While the raw JSON format contains extensive additional details (such as tactical lineups and body-part attributes), the preprocessing pipeline isolates and extracts only the essential fields required for real-time replay.

To bypass the computational overhead and complexity associated with parsing large JSON structures at runtime, the `event_reader.py` script was developed. The parsing algorithm utilizes Python's native `json` library to filter the raw match data, isolating exclusively the essential attributes required for the replay system.

```
5411   00:05   Pass    44.8    5.0   Mikel Balenziaga Oruesagasti -> Inigo Martinez
    Berridi
6454   00:06   Ball Receipt*   33.3    14.1    Inigo Martinez Berridi
6454   00:06   Carry   33.3    14.1    Inigo Martinez Berridi
8905   00:08   Pass    33.3    14.1    Inigo Martinez Berridi -> Mikel Balenziaga
    Oruesagasti
```

Listing 2: Sample processed event stream ready for the C replayer server.

As illustrated in Listing 2, each line in the resulting plain-text file corresponds to a single match event. The attributes are delimited by tab characters (`\t`), precisely adhering to the fields specified in the project requirements.

Analyzing a sample of 5 different matches, it is observed that an average of approximately 3,562 events are recorded per match. The total number of events varies depending on the pace of each game, ranging from 3,205 to 3,910 events in our sample. As mentioned, these events do not have a constant recording rate; rather, they are distributed over time based on the actual actions (e.g., passes, shots) performed with the ball. This is why there is often a gap of a few seconds in the regular flow (e.g., the clock might jump from 37:10 to 37:12). This occurs because the available data is strictly event-driven, meaning time elapses without any new records until the next distinct action takes place.

1.2 The Replayer Thread and Shared Memory

The primary objective of this thesis is the playback and transmission of the aforementioned parsed data to multiple clients via a server. To achieve this, the Replayer thread (replayer_thread_function) was developed, which is responsible for reading the file and writing the events into a shared memory structure (shared data). From there, the system subsequently handles their transmission to the connected clients. During this writing process, the Replayer enriches the data by adding two critical fields at the beginning of each event: the Sequence Number and the Due Time. The Sequence Number is an incremental identifier (1, 2, 3...) that denotes the correct sequential order of the events. This indicator is utilized by specialized test clients to detect any lost packets (gaps) or out-of-order deliveries. The Due Time, expressed in nanoseconds, specifies the exact system clock time (CLOCK_MONOTONIC) at which each respective event is due to be sent. This value is fundamental for system measurements, as the difference between the arrival time at the client and the due time constitutes the delivery latency.

For the aforementioned architecture to function correctly, the proper implementation of the shared memory is critical. The shared memory is represented by a `struct` named `MatchData`, which consists of the following fields:

- `char events[MAX_EVENTS][MAX_LINE_LEN]`: A two-dimensional array storing all the parsed events of the match.
- `int event_count`: An integer counter indicating the total number of events that have been appended to the memory so far.
- `bool match_finished`: A boolean flag indicating whether the playback of the match has concluded.
- `pthread_mutex_t mutex`: Given that the storage structure is a shared resource between the Replayer (producer) and the client-handling threads (consumers), access to it is protected via a mutual exclusion mechanism (mutex).
- `pthread_cond_t new_event`: A condition variable used by the Replayer to "wake up" the waiting client threads immediately after a new event is added to the shared list.

Below is a diagram that represents the exact method that the replayer thread follows in order to upload an event to the shared memory.

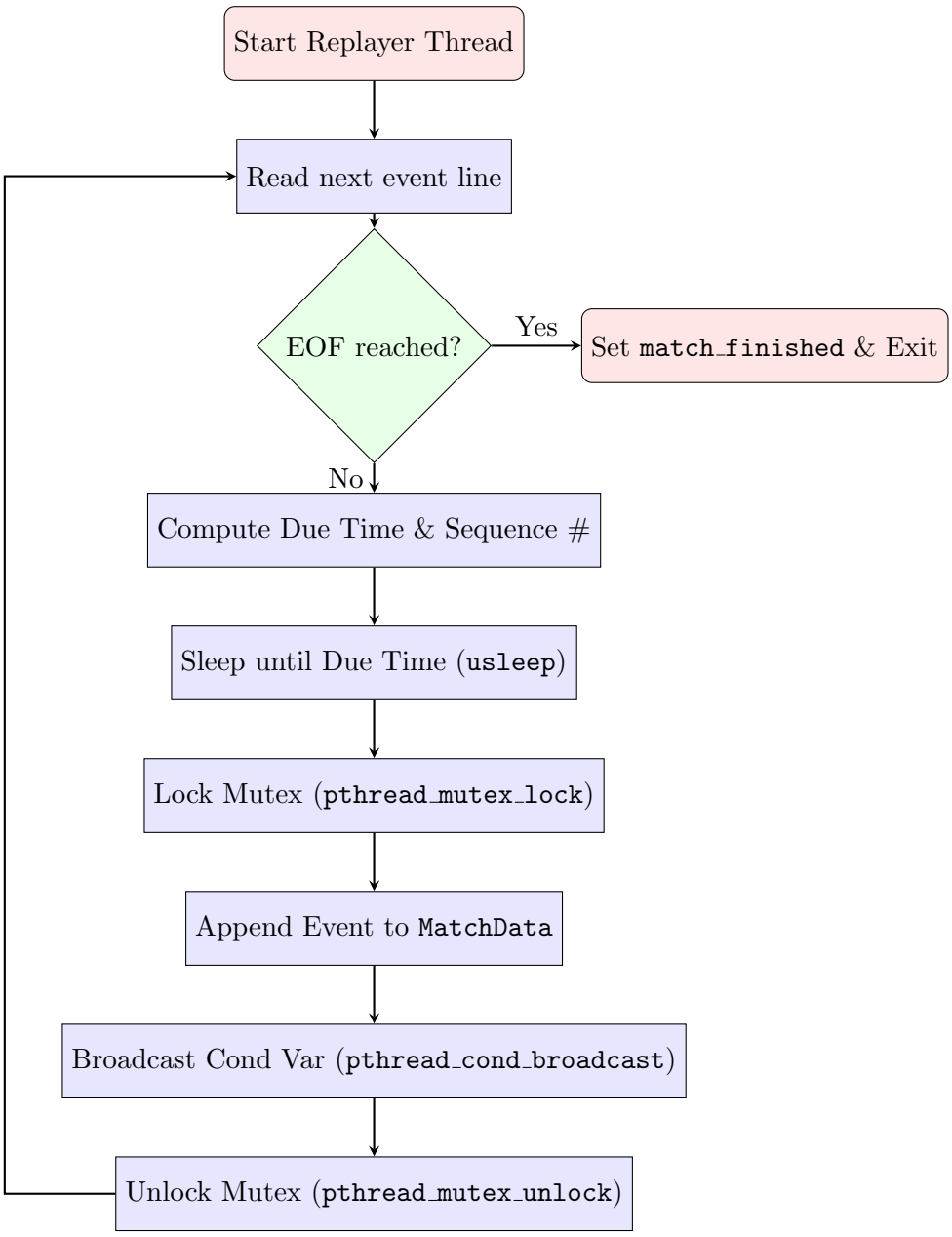


Figure 1: Flowchart of the Replayer thread execution loop.

1.3 Due Time Calculation and Time Synchronization

To simulate a real-time playback environment, each event is assigned a precise deadline, referred to as the `Due Time`. The Due Time specifies the exact system clock timestamp at which a given event must be made available to connected clients[**?**].

1.3.1 Mathematical Formulation

When the replay server initializes, it captures the baseline timestamp of the system clock using the monotonic clock source, denoted as T_{start}. For any event i occurring at M_i milliseconds from the match kick-off, its absolute Due Time ($T_{\text{due},i}$), expressed in nanoseconds, is calculated as follows:

$$T_{\text{due},i} = T_{\text{start}} + \left(\frac{M_i \times 10^6}{S} \right) \tag{1}$$

where:

- T_{start} is the initial monotonically increasing system timestamp (in nanoseconds) recorded when the replay begins.
- M_i represents the relative offset of the event from the start of the match (in milliseconds).
- 10^6 converts the millisecond offset into nanoseconds.
- S is the user-defined speed multiplier (e.g., $S = 1$ for real-time playback, $S = 60$ for 60× acceleration, or $S = 1000$ for high-throughput stress testing).

1.3.2 Clock Selection: Monotonic vs. Real-Time

A critical design decision in systems performance evaluation is the choice of clock source. The implementation strictly utilizes `CLOCK_MONOTONIC` via POSIX `clock_gettime()` rather than `CLOCK_REALTIME`[**?**]. Unlike wall-clock time (`CLOCK_REALTIME`), which is subject to NTP (Network Time Protocol) adjustments, leap seconds, or manual system time modifications, `CLOCK_MONOTONIC` provides a strictly increasing time measure since an unspecified system boot point. This guarantees that duration measurements remain strictly positive and linearly consistent across execution runs.

1.3.3 Latency Evaluation Metric

The embedded `Due Time` serves as the primary ground truth for evaluating system performance. Because the server and the load generator execute on the same host machine, they share a single unified clock reference[**?**]. Upon receiving an event line over the socket, the load generator immediately queries `CLOCK_MONOTONIC` to record the arrival timestamp (T_{arrival})[**?**]. The delivery latency L_i for event i is directly computed as:

$$L_i = T_{\text{arrival},i} - T_{\text{due},i} \tag{2}$$

A positive latency value ($L_i > 0$) indicates that the event reached the client behind schedule, reflecting system queueing delays or socket buffering overloads.